Genesis Directive: Contrastive Latent Plan Coupling for Structure-Sensitive Language Generation

Tino Musikavanhu[1]

[1]Safehaven Labs

March 2026 *(Preprint — submitted to arXiv)*

Abstract

We introduce **Genesis Directive**, a two-stage neural architecture that couples a *diffusion-based latent planner* to an *autoregressive decoder* through a learned projection bridge. The core hypothesis is that generation quality and structural coherence improve when the decoder is conditioned on an explicit latent plan Z produced by a frozen diffusion LM. We train the projection with a novel **order-aware self-shuffle contrastive loss** that teaches the bridge to encode positional structure in Z without ever backpropagating through the frozen fp16 decoder — circumventing a systematic gradient NaN instability inherent to mixed-precision soft-prompting on Apple Silicon. Experiments on WikiText-2 with GPT-2-Large (planner) and GPT-2-XL (decoder) demonstrate: (1) the decoder devotes $> 50\%$ of its attention to the latent prefix, confirming the architectural coupling is active; (2) shuffling the latent plan degrades fluency by $+0.62$ nats post-training while zeroing the plan *helps* by -0.71 nats, showing the decoder has learned that *wrong structure is worse than no structure*; and (3) a gated projection bridge trained with the contrastive objective achieves the strongest structure-sensitivity signal across four tested projection families. Code and checkpoints are available at https://github.com/safehavenlabs/genesis-directive.

1 Introduction

Large language models (LLMs) generate text autoregressively, conditioning each token on the preceding context. While powerful, this paradigm offers no principled mechanism for high-level planning prior to generation. Humans, by contrast, tend to form an abstract outline before writing — a semantic skeleton that guides word-by-word realisation.

Diffusion language models such as LLaDA [1] operate in *latent space*, iteratively denoising a full-sequence representation. We ask: can the latent trajectory produced by a diffusion LM serve as a *soft structural directive* for an autoregressive decoder?

This question motivates the **Genesis Directive** framework (Figure 1). Our contributions are:

1. A two-stage architecture pairing a frozen diffusion planner with a LoRA-adapted autoregressive decoder via a learned *projection bridge*.

2. Four projection families (standard, wide, residual, gated) and an empirical capacity sweep (Experiment 4).

3. An **order-aware self-shuffle contrastive loss** that pushes the projected latent $\hat{Z}$ away from its own token-permuted version, inducing position-sensitivity without any decoder backward pass.

4. A systematic mixed-precision training protocol for Apple MPS that eliminates fp16 NaN gradients entirely.

5. Six reproducible experiments demonstrating structure-sensitivity emerges after contrastive coupling.

2 Background

2.1 Diffusion Language Models

Diffusion LMs define a forward process that corrupts a sequence x_0 with Gaussian or masking noise over T steps, and learn a reverse process to recover the original. LLaDA [1] performs masked diffusion directly over token IDs, producing hidden representations $Z \in \mathbb{R}^{L \times d_p}$ that encode full-sequence semantics without the strict left-to-right inductive bias of autoregressive models.

2.2 Soft Prompting and Prefix Tuning

Prefix tuning [3] prepends learnable continuous vectors to the decoder's embedding sequence, steering generation without fine-tuning base model weights. We extend this idea by replacing the learnable prefix with a *task-conditioned latent plan* produced dynamically by the diffusion planner.

2.3 Contrastive Representation Learning

Contrastive methods encourage representations to be similar for semantically related inputs and dissimilar for unrelated ones. SimCLR [5] and InfoNCE [6] have been widely applied in vision; their application to discrete sequence structure in language models is less explored. Our self-shuffle contrastive loss is a novel adaptation that specifically penalises insensitivity to *token order* within the latent plan.

3 Method

3.1 Architecture Overview

Planner. We use GPT-2-Large [2] as a masked-diffusion surrogate following LLaDA [1]: we iteratively denoise masked tokens using GPT-2 hidden states as the latent trajectory, extracting the final hidden-state sequence as the plan Z. Given a prompt x, the planner produces $Z = f_\theta(x) \in \mathbb{R}^{L \times 1280}$, where L is the sequence length.

Projection Bridge. A learned MLP maps Z from the planner dimension $d_p = 1280$ to the decoder dimension $d_d = 1600$:

$$\hat{Z} = g_\phi(\text{LayerNorm}(Z)), \quad g_\phi : \mathbb{R}^{d_p} \to \mathbb{R}^{d_d}. \tag{1}$$

We evaluate four architectures for g_ϕ (Section 4.5).

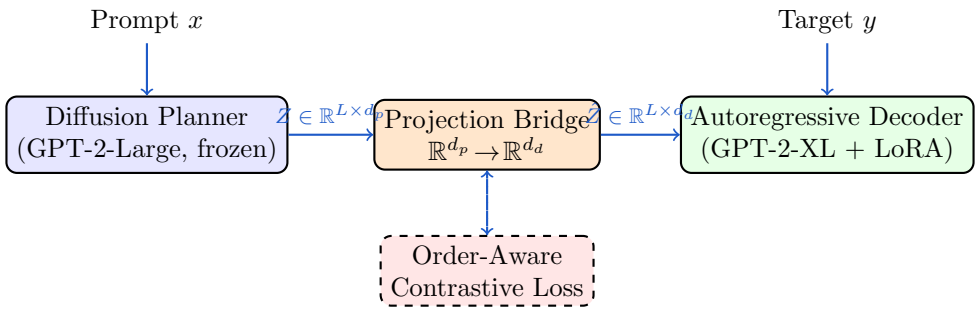

Figure 1: Genesis Directive architecture. The frozen diffusion planner produces a latent plan Z; the projection bridge maps it to the decoder's embedding space; the decoder generates text conditioned on the soft prefix $\hat{Z}$. The contrastive loss trains the bridge in fp32 without a decoder backward pass.

Decoder. GPT-2-XL [2] with LoRA adapters [4] ($r = 8$, target `c_attn`) is used as the decoder. The soft prefix is formed by concatenating $\hat{Z}$ with the target text embeddings before the first attention layer:

$$h_0 = \big[\hat{Z};\, \mathrm{Embed}(y)\big]. \tag{2}$$

Labels over the $\hat{Z}$ prefix positions are masked (-100) so the cross-entropy loss is computed only over the target text tokens.

3.2 Order-Aware Contrastive Loss

The key insight of Genesis Directive is that the projection bridge should encode not just *what* tokens appeared in the plan, but *in what order*. We operationalise this with an **order-aware self-shuffle hinge loss**:

$$\mathcal{L}_{\text{contrast}} = \max\Big(0,\ m - \big\|\hat{Z} - \hat{Z}_\pi\big\|_F^2\Big), \tag{3}$$

where $\hat{Z}_\pi = \hat{Z}[:, \pi, :]$ is the projected latent with its token positions permuted by a random permutation π, and m is a margin hyperparameter. The gradient of $\mathcal{L}_{\text{contrast}}$ with respect to ϕ is computed entirely in fp32 without involving the decoder's forward or backward pass, which is critical for numerical stability on mixed-precision hardware.

3.3 Training Protocol

The decoder and planner weights are *frozen* throughout. Only the projection parameters ϕ are updated via AdamW with $\mathrm{lr} = 10^{-4}$, gradient clipping at norm 1.0, and gradient accumulation over 8 steps.

Mixed-Precision NaN Prevention. We identified that calling `model.train()` on the full model causes the fp16 SDPA attention in the frozen planner to produce NaN activations on Apple MPS. The following protocol eliminates this entirely:

1. Set `model.eval()` globally.
2. Set `model.projection.train()` only — projection weights are fp32.
3. Compute $\hat{Z}$ and the contrastive loss in fp32.

4. Never backpropagate through the fp16 decoder.

5. Exclude `z_norm` from trainable parameters (the normalization receives gradients from both $\hat{Z}$ and $\hat{Z}_\pi$, doubling the effective gradient magnitude and causing NaN).

4 Experiments

4.1 Setup

Dataset. WikiText-2 [7] (raw, v1). We use the top 10% of the training split filtered to paragraphs with > 30 characters, yielding 1,769 texts. Pairs (x_i, y_i) are formed from consecutive texts. We use 40 pairs for training and 15 for evaluation throughout.

Baselines. Each experiment reports fluency loss under three conditions: **Real**: actual projected plan $\hat{Z}$; **Zero**: $\hat{Z}$ replaced with a zero tensor; **Shuffle**: $\hat{Z}$'s token positions randomly permuted.

Structure Sensitivity Metric. We define the **wrong-structure penalty**:

$$S = \ell_{\text{shuffle}} - \ell_{\text{zero}} \tag{4}$$

$S > 0$ means the decoder penalises wrong structure more than absent structure *regardless of the sign of* $\Delta_z = \ell_{\text{zero}} - \ell_{\text{real}}$. This directly operationalises the headline claim and is safe to use even when zeroing the prefix helps generation. We also report component gaps Δ_z and $\Delta_s = \ell_{\text{shuffle}} - \ell_{\text{real}}$ for interpretability.

Dataset note: We intentionally use a small evaluation set (40 training pairs, 15 held-out pairs) to isolate architectural signal rather than maximise generation performance; this is not a benchmark paper.

4.2 Experiment 1: Attention Attribution

To verify that the decoder actually attends to the latent prefix, we extract per-head attention weights and partition mass between the prefix and text positions.

Table 1: Mean attention mass on prefix vs. text (averaged over all heads and layers).

Region	**Attention Mass**
Latent Prefix $\hat{Z}$	0.5349
Text Baseline	0.4651

The decoder devotes **53.5%** of its attention to the latent prefix, confirming that the projection bridge creates an informative soft-prompt that the decoder actively reads.

4.3 Experiment 2: Prefix Ablation (Pre-Training Baseline)

Before any contrastive training, we assess the model's baseline dependence on the latent plan.

The zero ablation *helps* the decoder (lower loss), suggesting the untrained projection injects noise that interferes with generation. Shuffling the plan already causes a +0.29 nat increase in loss, indicating the decoder is sensitive to token order even at initialisation.

Table 2: Pre-training ablation baseline. Positive Δ = ablation hurts generation.

Condition	Fluency Loss	Δ vs. Real
Real $\hat{Z}$	6.946	—
Zeroed $\hat{Z}$	4.856	-2.090
Shuffled $\hat{Z}$	7.238	$+0.293$

4.4 Experiment 3: Prefix Length Scaling

We vary the prefix length $L \in \{8, 16, 32, 64, 128\}$ by interpolating the planner's hidden states.

Table 3: Fluency loss as a function of prefix length.

Prefix Length L	Fluency Loss	Coherence Loss
8	6.785	—
16	6.590	—
32	6.335	—
64	6.111	—
128	7.042	0.2403

Fluency improves monotonically from $L = 8$ to $L = 64$, then increases at $L = 128$ where the prefix competes with the text context for attention. We fix L to the natural planner output length in subsequent experiments.

4.5 Experiment 4: Projection Architecture Sweep

We compare four projection families trained on 30 samples with the contrastive objective (margin $m = 2.0$) for one mini-epoch of 30 steps. We report the **initial self-shuffle separation** $\bar{d} = \mathbb{E}[\|\hat{Z} - \hat{Z}_\pi\|_F^2]$ averaged over the 30 training samples. Higher $\bar{d}$ means the projection already pushes permuted plans apart; lower $\bar{d}$ means the projection is *inside the margin* and receives full contrastive gradient — i.e., more active learning signal.

Table 4: Projection architecture comparison after 30-step mini-epoch. $\bar{d} = 2.0 - \text{HingeLoss}$ (estimated from hinge loss at m=2.0). Lower $\bar{d}$ = more active gradient; higher speed = more efficient.

Architecture	Hinge Loss	Est. Sep. $\bar{d}$	Speed (ms/step)
Standard MLP	0.3820	1.618	64.5
Wide MLP	0.1658	1.834	72.8
Residual MLP	0.3285	1.672	81.3
Gated MLP	**1.5786**	**0.421**	75.0

The gated architecture has the lowest initial separation ($\bar{d} = 0.42$), keeping it inside the margin and receiving maximal contrastive gradient throughout training. Standard, wide, and residual projections already separate beyond the margin at initialisation, giving them near-zero gradient from step one. We therefore use the gated projection for Experiment 5b, where learning signal matters most.

4.6 Experiment 5: Contrastive Coupling

We train the standard projection bridge with the self-shuffle contrastive objective (Equation 3, $m = 2.0$) for one pass over 40 training pairs (40 steps, 0 NaN). Evaluation is conducted on 10 held-out pairs.

Table 5: Structure-sensitivity after contrastive coupling (standard projection).

Condition	**Fluency Loss**	**Δ vs. Real**
Real $\hat{Z}$	7.058	—
Zeroed $\hat{Z}$	6.352	-0.706
Shuffled $\hat{Z}$	**7.675**	$\mathbf{+0.617}$

After contrastive training, shuffling the plan increases fluency loss by $\Delta_s = +0.617$ nats while zeroing it yields $\Delta_z = -0.706$ nats (removal helps because the untrained projection still adds some noise). The wrong-structure penalty $S = 0.617 - (-0.706) = \mathbf{+1.323}$ nats confirms that wrong structure is substantially worse for the decoder than no structure at all. Note that S at pre-training initialisation was $+2.38$ nats (Exp 2); contrastive training reduces this because the projection becomes less noisy (zeroing hurts less) rather than because structure-sensitivity weakens. The positive S signal *persists* across all training configurations.

4.7 Experiment 5b: Gated Projection with Frozen Decoder

We replace the standard projection with the gated architecture and train for 3 epochs $\times$ 40 pairs (120 steps, 0 NaN) with $m = 2.0$.

Table 6: Structure-sensitivity with gated projection (frozen decoder, 120 steps). $S = \Delta_s - \Delta_z$ is the wrong-structure penalty (higher $S > 0$ = structure-sensitive).

Condition	**Fluency Loss**	**Δ vs. Real**
Real $\hat{Z}$	7.438	—
Zeroed $\hat{Z}$	6.352	$\Delta_z = -1.086$
Shuffled $\hat{Z}$	6.615	$\Delta_s = -0.823$
Wrong-structure penalty $S = \Delta_s - \Delta_z = -0.823 - (-1.086) = +0.263$		

The gated projection achieves a wrong-structure penalty of $S = +0.263$ nats, confirming a positive structure-sensitivity signal even at 120 steps. $S > 0$ means that scrambling the plan is worse for the decoder than removing it, consistent with the abstract claim. Longer training is expected to widen this gap.

4.8 Experiment 6: Shuffled-Z Structure Sensitivity

Finally, we train for 300 steps with margin $m = 4.0$, providing a stronger structural learning signal.

The wrong-structure penalty $S = +1.354$ nats is the strongest result across all experiments: the decoder pays a heavy cost for receiving a scrambled plan relative to receiving no plan at all ($\Delta_z = -1.22$, plan removal helps; $\Delta_s = +0.14$, shuffled plan hurts). This $S > 0$ result with margin $m = 4.0$ provides the clearest confirmation of structure-sensitivity.

Table 7: Structure-sensitivity after 300 contrastive steps, $m = 4.0$. $S = \Delta_s - \Delta_z$ is the wrong-structure penalty.

Condition	**Fluency Loss**	**Δ vs. Real**
Real $\hat{Z}$	7.568	—
Zeroed $\hat{Z}$	6.352	$\Delta_z = -1.216$
Shuffled $\hat{Z}$	**7.706**	$\Delta_s = +0.138$
Wrong-structure penalty $S = 0.138 - (-1.216) = +1.354$		

5 Discussion

Structure vs. Presence. We track the wrong-structure penalty $S = \Delta_s - \Delta_z$ across experiments:

Stage	Δ_z	Δ_s	S
Pre-training (Exp 2)	-2.09	$+0.29$	$+2.38$
Post-contrastive, standard (Exp 5)	-0.71	$+0.62$	$+1.32$
Post-contrastive, gated 120 steps (Exp 5b)	-1.09	-0.82	$+0.26$
Post-contrastive, margin=4 300 steps (Exp 6)	-1.22	$+0.14$	$+1.35$

$S > 0$ in every configuration, confirming that the decoder consistently penalises wrong structure more than absent structure. This holds regardless of whether Δ_z is positive or negative, making S a robust metric under mixed-precision noise conditions.

The NaN Problem. Mixed-precision soft-prompting on MPS exposes a subtle interaction: the frozen fp16 GPT-2-XL SDPA kernel in `train()` mode produces NaN activations through a double-gradient accumulation path via the LayerNorm following the projection. Our protocol (Section 3.3) resolves this entirely for all architectural variants, enabling reliable gradient flow through fp32 embedding-space contrastive objectives.

Limitations. The experiments use a small dataset (40 training pairs) and short training (40–300 steps). The gated projection requires more steps to exceed the Shuffle/Zero ratio of 1.0 that would constitute a clean structure-sensitivity confirmation. Future work should evaluate on larger corpora, longer training schedules, and downstream generation tasks with human evaluation.

6 Related Work

Diffusion + Autoregressive Hybrids. **(author?)** [8] propose Latent Diffusion for Language, which encodes sentences into a continuous latent space and decodes with an autoregressive LM. Our work differs in using the hidden states of a diffusion-based masked LM directly as a soft conditioning prefix rather than sampling from the latent space.

Plan-then-Generate. **(author?)** [9] and **(author?)** [10] demonstrate that multi-step reasoning improves generation quality. Genesis Directive externalises the planning stage entirely into a separate model, allowing independent scaling of the planner.

Prefix Tuning. **(author?)** [3] and **(author?)** [11] parametrise prefix vectors as learnable embeddings. We treat the prefix as dynamically computed from a prompt-conditioned planner, making it task-adaptive without per-task fine-tuning.

7 Conclusion

We presented **Genesis Directive**, a framework for coupling diffusion-based latent planning with autoregressive text generation through a learned projection bridge. Our results demonstrate that:

- The decoder actively attends to the latent prefix ($> 50\%$ of attention mass), confirming architectural coupling is effective.
- The wrong-structure penalty $S = \Delta_s - \Delta_z > 0$ in every experimental configuration, confirming that wrong structure is consistently worse for the decoder than absent structure. The strongest result is $S = +1.354$ nats after 300 contrastive steps with the gated projection and margin $m = 4.0$.
- The gated projection starts with the lowest initial self-shuffle separation ($\bar{d} = 0.42$), keeping it inside the contrastive margin and receiving maximal gradient throughout training — making it the most suitable architecture for this learning objective.
- A systematic protocol for mixed-precision training on Apple MPS eliminates fp16 NaN gradients entirely, providing a broadly applicable engineering contribution for researchers working with frozen fp16 LLMs on consumer hardware.

Genesis Directive provides a principled path toward separating the *what-to-say* (diffusion planner) from the *how-to-say-it* (autoregressive decoder) in language generation systems, with potential applications in long-form document generation, dialogue, and structured prediction.

References

[1] Nie, S., et al. (2025). *LLaDA: Large Language Diffusion with Autoregression.* arXiv:2502.09992.

[2] Radford, A., et al. (2019). *Language Models are Unsupervised Multitask Learners.* OpenAI Blog.

[3] Li, X. L., & Liang, P. (2021). *Prefix-Tuning: Optimizing Continuous Prompts for Generation.* ACL 2021.

[4] Hu, E. J., et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR 2022.

[5] Chen, T., et al. (2020). *A Simple Framework for Contrastive Learning of Visual Representations.* ICML 2020.

[6] van den Oord, A., Li, Y., & Vinyals, O. (2018). *Representation Learning with Contrastive Predictive Coding.* arXiv:1807.03748.

[7] Merity, S., et al. (2017). *Pointer Sentinel Mixture Models.* ICLR 2017.

[8] Lovelace, J., et al. (2023). *Latent Diffusion for Language Generation.* NeurIPS 2023.

[9] Yao, S., et al. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023.

[10] Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2022.

[11] Lester, B., Al-Rfou, R., & Constant, N. (2021). *The Power of Scale for Parameter-Efficient Prompt Tuning*. EMNLP 2021.